

Automatic Fire: The Creation of the Doom Slayer's Shoulder Cannon

Alyssa Jackson

University of Colorado Boulder

Executive Summary

As computer science continues to advance and large language models (or LLM) get better at identifying objects, object detection and recognition gets faster and more available overall. Projects that utilize the quick identification of objects become easier to achieve for the “average” person and have a wider range of possibilities for computers to do more intricate things. One of these projects is using computer vision, face detection, a microcontroller, and two servo motors to create a face tracking system that has a “turret” point a laser at the last face it has detected on a video-streaming camera. This report will walk through that process and show how it was accomplished as well as how it can be remade.

Parts List and Physical Assembly

All the hardware needed to build the face-tracking turret is as follows:

- Laser pointer
- 2 Servo Motors
- 2 3D Modeled and Printed Stands
- 2 Machine Screws and Nuts
- Power Supply
- Main Computer
- Camera
- Arduino
- Breadboard
- Jumper Wires

Due to the inaccuracy of the size of the 3D printed stands, two extra spacers were also needed to make everything fit together. An updated 3d model will be created so the spacers will no longer be needed.



Figure 1. The built physical turret composed of the servo motors, the 3d printed stands, the machine screws, nuts, and the spacers. The laser pointer is not attached here.

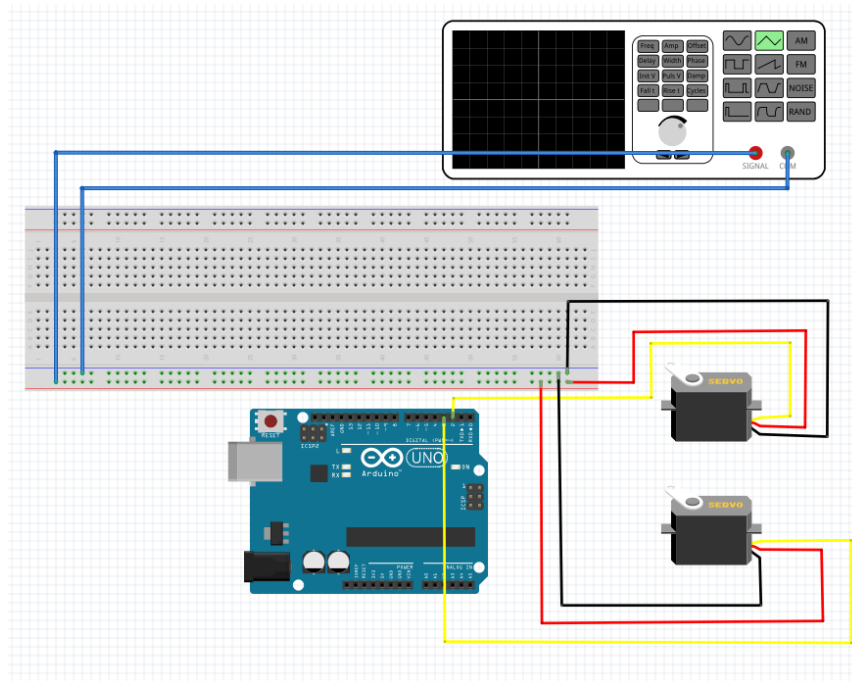


Figure 2. The circuit diagram of the completed turret circuit.

Figure 1 shows the built turret made up of the parts listed in the caption. To put it totally, the arm of the servo that matches the holes of the stands are fitted in, the servo is inserted on the opposite side of the arm, and the screw that holds the arm to the motor are screwed in. The stand that holds

the y-axis servo is screwed into the bracket tabs of the x-axis servo with the machine screws and the nuts. No special stand was made for the laser pointer to attach to the y-axis servo, so tape was used to both hold the laser in place and to keep the button pressed.

The wiring diagram is in Figure 2. Since the current draw from the two servo motors is too large for the 5V pin on the Arduino, a separate power supply was needed for the project to work properly. The power and ground from the power supply is connected to the corresponding power and ground pins of the servos. The signal pin from the x-axis servo connects to digital pin 2 from the Arduino and the y-axis one is connected to digital pin 3. The Arduino is then connected to the main pc that will be running the Python script, and with that the physical turret and connections are made and ready to run.

Coding and the Software Side

Setting up the coding side of the project is a bit more complicated, especially if the user doesn't have an IDE or any sort of way to run a Python script. So for the sake of this part of the guide, it will be assumed that the user has already downloaded Python and has already set up a reliable method for running Python scripts. The most important part for running this project is to download the OpenCV library for Python. OpenCV is a free open-source library for all things computer vision, or the field that focuses on programming computers to see and interpret visual information. After downloading OpenCV in either a virtual environment or on all of Python entirely, the corresponding library can be used.

In order to run the Arduino code, the Arduino IDE needs to be installed and the Servos library needs to be downloaded from the IDE's in-app library manager tab. Once set up, both the Python script (Appendix A) and the Arduino code (Appendix B) are ready to be downloaded from the aJax-EXE Personal_Projects GitHub repository (Appendix C) and run on the corresponding computers.

Haar Cascades

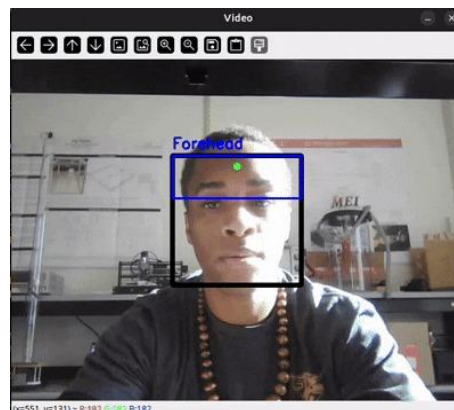


Figure 3. An image of the simulation of how the laser should point at the forehead of the target. This image also shows how the face detection of the Python script.

The method used to detect the faces in the project, as seen in Figure 3, is called Haar Cascade. The modern algorithm was developed by Paul Viola and Michael Jones, but the math behind it was developed by mathematician Alfred Haar. The principle behind it is that Haar features that commonly occur in all objects are scanned for and identified.

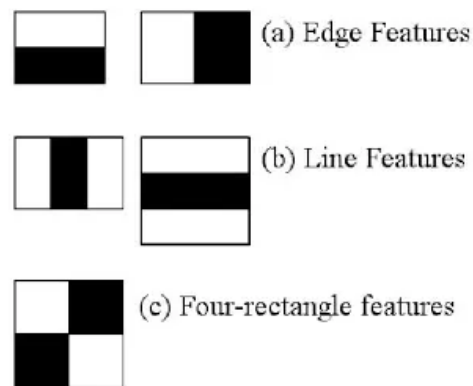


Figure 4. Common Haar features that are looked for in an image when using Haar Cascades.

As can be seen in Figure 4, these seemingly random patterns are identified within an image at varying sizes and orientations. Once they are identified, they become known as weak classifiers. Most of these classifiers are ignored, but for the ones that do stay, they are used in a process called Adaboost Train which then takes these weak classifiers and combines them together to form strong classifiers. It is with these strong classifiers that objects are categorized with and grouped together to form the dataset which a program can reference in order to identify an object with. If that object does not have a dataset of these classifiers (trying to find a dog with a cat dataset), or the object is too unique to be identified or just outside the dataset (turning your head with the frontal face identifier, having scars, etc.), the object will not be identified. Because the scope of this project only needs to use the front of the face, false negatives are not common but aren't nonexistent either.

Conclusion and Results

While the results of the code can't be shown in just a Word document due to project being video-based, the project worked very well. The servo motors did move in time with the detection and movement of the face, but the main issue was trying to get the laser to point exactly at the forehead. The position of the webcam of the laptop was too easy to move in and out of sync with

the movement range of the turret. This would cause the laser to point too high or too low sometimes.

Because this project was a personal project that was started on before not as an assignment, there are many things that would be changed or further researched into to make improvements to. For example, this project was originally meant for a Halloween costume, so having a better 3d model with a more consistent positioning and calibration method would be ideal. More research could be done to look into using the Intersection of Union in order to combine the datasets from the frontal face and the profile of the face to get a higher target detection rate and time. With this project, and with object detection in general, the potential for customization and personalization is seemingly endless. When combined with face recognition, other object detection datasets, and other electrical components like lights, lasers, and sounds, then a wide array of outputs and outcomes can occur and bring endless possibilities for this project in the future.

Appendix A – Python Script

```
import cv2
import numpy as np
import serial
import time

# Load the Haar cascade file for face detection
face_cascade = cv2.CascadeClassifier('Equipment Launcher/Haar Cascade
Classifiers/haarcascade_frontalface_default.xml')

# Setting up the serial communication to the Arduino
arduinoData = serial.Serial('/dev/ttyACM0', 9600, timeout=0.01, write_timeout=0.01)

LAST_SEND = 0
SEND_INTERVAL = 0.10 # only 10 msgs/sec

def send_coordinates_to_arduino(x, y):
    # Convert the coordinates to a string and send it to Arduino
    global LAST_SEND
    now = time.time()

    if now - LAST_SEND < SEND_INTERVAL:
        return

    coordinates = f"{x},{y}\r"

    try:
        arduinoData.write(coordinates.encode())
    except serial.SerialTimeoutException:
        print("Serial timeout - Arduino overloaded")
    except Exception as e:
        print("Serial error:", e)

    LAST_SEND = now

def main():
    # Setting up the webcam camera feed; 0 is webcam, 1 is an external camera
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
```

```
print("Error: Could not open webcam.")
return
while True:
    # Checks if the camera feed is running or not
    ret, frame = cap.read()
    if not ret:
        print("Error: Could not read frame.")
        break
    grey = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(grey, 1.05, 8, minSize=(120,120))
    # Drawing the Rectangle around the last face
    if len(faces) > 0:
        # Get the coordinates of the last detected face
        (x, y, w, h) = faces[-1]

        # Find the rough height of the forehead
        forehead = h//3

        # Drawing the rectangle and labeling the face
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(frame, "Target Face", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (36, 255, 12), 2)

        # Getting the coordinates for the Center(ish) of the Forehead
        x_center = x+w // 2
        y_center = y+forehead // 4
        send_coordinates_to_arduino(x_center, y_center)
        cv2.imshow('Face Detection', frame)
        if cv2.waitKey(10) & 0xFF == ord('q'):
            break
        cap.release()
        cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

Appendix B – Arduino Code

```
#include <Servo.h>

Servo left_right;
Servo up_down;

String buffer = "";
unsigned long lastMoveTime = 0;

int targetX = 90;
int targetY = 90;

const int MOVE_INTERVAL = 20; // ms between small servo steps (smooth movement)
const int STEP_AMOUNT = 2; // how much servo changes per step

void setup() {
  left_right.attach(2);
  up_down.attach(3);

  Serial.begin(9600);
}

// Non-blocking serial read
void readSerial() {
  while (Serial.available()) {
    char c = Serial.read();

    if (c == '\r') {
      // parse full command
      int commaIndex = buffer.indexOf(',');

      if (commaIndex > 0) {
        int x_axis = buffer.substring(0, commaIndex).toInt();
        int y_axis = buffer.substring(commaIndex + 1).toInt();

        // Convert to servo angle
        targetY = map(y_axis, 0, 1080, 180, 0);
        targetX = map(x_axis, 0, 1920, 180, 0);

        // Constrain
        targetX = constrain(targetX, 0, 180);
```

```
targetY = constrain(targetY, 0, 180);
}

buffer = ""; // clear after parsing
}
else {
buffer += c; // build message
}
}
}

// Smooth servo motion (prevents current spikes)
void updateServos() {
unsigned long now = millis();
if (now - lastMoveTime < MOVE_INTERVAL) return;
lastMoveTime = now;

int currentX = left_right.read();
int currentY = up_down.read();

// Smooth easing toward target
if (currentX < targetX) currentX += STEP_AMOUNT;
else if (currentX > targetX) currentX -= STEP_AMOUNT;

if (currentY < targetY) currentY += STEP_AMOUNT;
else if (currentY > targetY) currentY -= STEP_AMOUNT;

currentX = constrain(currentX, 0, 180);
currentY = constrain(currentY, 0, 180);

left_right.write(currentX);
up_down.write(currentY);
}

void loop() {
readSerial();
updateServos();
}
```

Appendix C – GitHub Link

https://github.com/ajax-EXE/Personal_Projects